

Local Autocomplete with llama.cpp — Write-up

- **Machine:** Apple M2 (8-core, 4P+4E), 16 GB unified memory, macOS 26.5.
- **Runtime:** llama.cpp build b9654 (prebuilt macOS arm64, Metal; native gemma4 arch support), via llama-server.
- **Model:** Google **Gemma 4 E2B** (gemma-4-E2B), **Q4_K_M** GGUF (around 3.2 GB), run as both the base and instruction-tuned (-it) variants.
- **Loop:** walk a realistic work email; at 21 cut points, feed the growing prefix to the model and ask for the continuation (harness.py). Each request is streamed so we capture time-to-first-token (TTFT). All numbers below are measured on this machine; the harness emits the JSON in results/.

1. Model choice — I picked the base gemma-4-E2B, not -it

Autocomplete is a continuation task: given the text so far, predict the next tokens in the user's own voice and format. That is literally the next-token objective the base model is trained on. The instruction-tuned -it model is trained to behave like a chat assistant: take an instruction, respond, stop. Using it for raw continuation feeds it text with no template, i.e. out-of-distribution, so it can flip into assistant mode or emit an end-of-turn token early.

I ran both on identical prefixes (configs A vs G, full set in EXAMPLES.md):

typed so far	base	-it
...report yesterday. I went	through it and made some comments.	through it and have some initial thoughts. Overall, it'...
...and I think it	looks great. I'm not sure if you've seen...	's a solid start. I have a few minor suggestions, mainly...
...before we send it to the	board.	final approvers. Firstly, I think we could strengthen t...

Difference: on Gemma 4 both variants are fluent. The difference is: the base model continues as the person drafting the email; the -it model adopts a helpful-reviewer stance ("I've had a chance to review it", "a few minor suggestions"). For an inline suggestion we want the former. Combined with the in-distribution argument and the risk of early end-of-turn stops, base is the right default.

Note the metric does **not** decide this: word-match is essentially tied (base 0.67 vs -it 0.71, within noise). The choice is about voice/format fit and being in-distribution, not about matching the original wording. Both variants are the same size and run at the same speed (A vs G TTFT p50 231 vs 267 ms), so there is no latency reason either way.

2. Latency — where the time goes, and what I did about it

Every request splits into two phases the server times separately:

- **Prefill:** encode the prefix into the KV cache with compute-bound.
- **Decode:** generate suggestion tokens one at a time with memory-bandwidth-bound (one full pass over the weights per token).

$TTFT \approx \text{prefill} + 1 \text{ decode step}$; $\text{total} \approx \text{prefill} + n_{\text{predict}} \times \text{decode_step}$.

Measured rates (M2, Metal, Q4_K_M): decode $\approx 38 \text{ tok/s}$ (around 26 ms/token), prefill $\approx 409 \text{ tok/s}$ (around 2.4 ms/token). So each generated token costs around 15× a prefilled one. Therefore, generation dominates total latency.

Experiment matrix (median over the 21-request loop, email passage)

config	TTFT p50	TTFT p90	total p50	prefill p50	decode p50	gen toks
A base, Metal, cache on, greedy, n=24	231 ms	386 ms	713 ms	228 ms	537 ms	20

config	TTFT p50	TTFT p90	total p50	prefill p50	decode p50	gen toks
B base, Metal, cache OFF, n=24	235 ms	386 ms	720 ms	233 ms	529 ms	20
C base, Metal, greedy, n=8	233 ms	342 ms	415 ms	229 ms	169 ms	8
D base, Metal, n=48 + stop at blank line	274 ms	339 ms	420 ms	237 ms	161 ms	12
E base, CPU only (ngl=0) , n=24	692 ms	1242 ms	1373 ms	691 ms	736 ms	21
G -it, Metal, greedy, n=24	267 ms	356 ms	684 ms	265 ms	486 ms	21
H base, Metal, greedy, n=24, mid-cursor	228 ms	328 ms	718 ms	225 ms	520 ms	22

(Config H is the **edit-in-the-middle** run; its latency is in line with A — same model and decode budget — but its point is the *quality* failure in Q4, not speed.)

What brought latency down (and what didn't)

1. GPU offload (Metal), -ngl 99 — the biggest win for prefill.

A vs E: prefill **409 vs 135 tok/s (3.0×)**, TTFT 231 → 692 ms without it. Decode improves only 1.3 times (38 vs 28 tok/s): decode is bandwidth-bound and the M2's GPU and CPU share the same unified memory, so the GPU mainly helps the compute-heavy prefill.

2. Cap the suggestion length — the biggest win for total latency.

A→C (n_predict 24→8) cuts total **713 → 415 ms**, because decode time is linear in tokens generated. For Smart-Compose we only show a few words, so a short cap is almost free on quality and nearly halves latency.

3. Prefix caching — expected to be the big win, but it isn't here.

As the user types, request $N+1$ extends request N , so the prefix KV should be reused and prefill should stay flat. It doesn't: A (cache on) vs B (cache off) are within noise (228 vs 233 ms), and prompt_n (tokens actually re-encoded) grows almost linearly with prefix length. I traced it to find the reason (diagnose_cache.py): the cache does work for an exactly-repeated prompt (resend drops prefill **526 → 61 ms**), it's not the generated tokens (n_predict=1 vs 24 give identical prompt_n), it's purely prefix length. The server log shows the cause: n_swa = 512 and `invalidated context checkpoint ... n_swa = 512`. **Gemma 4 E2B uses interleaved sliding-window attention (window 512)**; the SWA layers invalidate the KV checkpoints prefix-reuse relies on, so the growing prefix is re-encoded almost every keystroke.

4. Flash attention -fa on, threads -t 4 (P-cores), --no-warmup — kept on, small effects.

Bottom line: a good config (base, Metal, greedy, n_predict≈8, stop at boundary) gives **TTFT in around 230 ms and a full short suggestion in around 415 ms**. TTFT decides "instant"; around 230 ms behind a typing debounce is fine, and it is prefill-bound, so it will creep up on long documents until caching is fixed.

3. Accuracy — how good, how measured, how to push further

How I measured it:

- Automatic: the passage's own continuation is a known-good next text, so I count how many leading words of the suggestion match what the author actually wrote (word_match). Strict **lower bound** — many continuations are valid, and matching the exact original wording is harsh. Result: **mean 0.67 leading words exact-match** (base, config A), with a long tail that does match several words; the metric undersells real quality.
- Qualitative: every raw suggestion is dumped (EXAMPLES.md), and they are

genuinely good: "I went" → "through it and made some comments."; "The summary" → "is clear and concise, and the data is well-presented."; "send it to the" → "board." In-voice and on-topic; they just pick a different valid word, which exact-match punishes.

The decoding step. Once the model has a probability distribution per position, decoding turns it into emitted tokens, and there's a lot to control:

- **Greedy vs sampling / temperature.** For autocomplete we want the safest, highest-probability continuation, so **greedy (temp = 0)** is the right default: best acceptance odds, deterministic (cacheable, testable). Sampling (temp 0.7, config F) adds diversity we don't want; on my noisy metric it happened to score higher (0.81 vs 0.67) but, it may also makes occasional confidently-wrong guesses.
- **Truncation: top-k / top-p / min-p.** Even at low temperature, clip the tail so a rare bad token can't slip in.
- **n_predict + stop tokens (length is a decoding choice).** Generating to a sentence end / blank line and stopping (config D) makes suggestions feel complete and, per Q2, is also the main latency lever — **the rare knob that improves quality and latency together**. Showing one clause beats dumping a paragraph.
- **Repetition / n-gram penalties** to stop the model echoing the prefix or looping.
- **Confidence gating** Read the top-token probability; if it's low, show nothing.
- **Higher-cost options:** speculative decoding (a tiny draft model proposes, E2B verifies — same quality, faster, so it *buys back* latency); beam search (better sequences but multiplies decode cost — not worth it for a few words); a higher-bit quant for marginal quality at more memory/bandwidth.

Tradeoff summary: the quality wins that matter most for autocomplete (greedy + truncation + stop-at-boundary + confidence gating) are **free or latency-positive**. The only genuine quality-vs-latency tension is beam search / larger quant (I'd skip), and speculative decoding (strictly good if you can fit a draft model).

4. Where this model falls short — the cursor in the middle

Everything above assumes the cursor is at the **end**. The base gemma-4-E2B is a standard **left-to-right causal** model: it conditions only on tokens *before* the cursor. With text on both sides (the user went back to fix an earlier sentence), it will **ignore everything after the cursor** — so it can suggest something that duplicates, contradicts, or runs into the following paragraph. It has no way to aim at the suffix.

`harness.py --mid-cursor` replays the same email but treats each cut point as an *nsertion point with text already to its right: it still sends the model only the prefix and then scores the suggestion against the already-written suffix.

Overlap is now the **failure** signal — it means the model re-proposed text the author already has just past the cursor.

metric (base, greedy, n=24, 19 mid-cursor points)	value
suffix collision rate (suggestion duplicates / runs into the suffix)	0.47
leading suffix words duplicated, mean	0.68
3-gram verbatim re-use of the near suffix	0.11

So on **~47% of mid-document edits** the suggestion collides with what's already there. The metric is a lower bound — it onlycatches verbatim collisions, not semantic one.

What we'd want instead:

- A model trained with a **fill-in-the-middle (FIM) / infilling** objective, which uses special tokens to reorder (*prefix, suffix* → *middle*) so it genuinely conditions on both sides — standard for code completion (CodeGemma, StarCoder, DeepSeek-Coder). A plain text model needs the same training to do this well. (I did not confirm whether Gemma 4 E2B ships FIM tokens; the base GGUF I used behaves as pure left-to-right — confirmed by config H above — so treat mid-text as unsupported unless verified.)
- A **cheap workaround without changing models:** put the suffix into the prompt as context and ask for the bridging text (e.g. "*continue so it flows into: «suffix»*"). But that needs the `-it` model (instruction-following), costs extra prefill, and is less reliable than a real FIM model. So mid-text editing is the one case where `-it` earns its

place.

For end-of-text Smart-Compose, base E2B is a good fit; for true mid-document editing it's the wrong tool and I'd reach for a FIM-trained model.

5. Production — what I'd do differently shipping to many people

Latency & the loop

- **Debounce keystrokes** (around 150–250 ms after a pause) and **cancel the in-flight request** when a new keystroke arrives — don't pay for suggestions the user is already typing past.
- **Fix prefix caching for the SWA model** (the long-document problem from Q2): keep

a **per-document persistent KV/slot** so the prefix is encoded once and extended; or window the context the model sees (last N tokens) so prefill stays bounded; or pick a model whose attention allows clean reuse if that matters more than SWA's memory savings.

- **Speculative decoding** with a tiny draft model to push decode past ~38 tok/s.
- **Keep the model resident and warmed** (`mlock`); never pay model-load latency on a keystroke. Ship the right backend per platform (Metal/CUDA/Vulkan/CPU) and pick the quant by device RAM (Q4_K_M needs ~3.5 GB; offer a smaller quant on 8 GB machines).

Quality & trust

- **Acceptance rate is the real metric.** Instrument shown-vs-accepted, segment by

context, and tune thresholds/temperature against it instead of an offline proxy.

- **Trigger heuristics:** only suggest at word/sentence boundaries; ****suppress in**

passwords, code, and PII-ish fields**;

- **Confidence gating** (Q3): show nothing when the model is unsure.
- **Safety:** the model can emit biased / unsafe / hallucinated continuations of

private text — add output filtering, and keep everything **on-device** (the privacy premise): no prefixes leave the machine, including in telemetry (log acceptances, not content).

Engineering

- Bound memory/CPU so autocomplete doesn't starve the host app; watch thermals/battery (sustained inference throttles); add a graceful **no-suggestion fallback** when the model is busy or the device is constrained.
 - A/B model + decoding configs behind flags; pin the llama.cpp build and model hash
-